Department of
Computer Science and Engineering
University of Dhaka

Lab Assignment 4

ADC Multi-Resolution Testing with Flash-Based Data Logging

CSE 2206:
Microcontroller and Embedded System Lab
2nd Year 2nd Semester, 2025

Date of submission: 27 July 2026

Group No: GB-10

Submitted to:
Dr. Mosaddek Hossain Kamal Tushar, Professor
Dept. of CSE, University of Dhaka

Submitted by:
Md. Samiul Islam Siam (02)
Partho Kumar Mondal (07)

Table of Contents

Test Case Results

SL	Test Case	Steps	Expected Result	Actual / Observed	Verdict
TC1	Raw ADC responsiveness	Rotate POT fully min→max while printing raw code	Raw code changes monotonically, roughly 0 to ~4095	0 to 4095 (12 bit res)	Pass
TC2	Flash erase verification	Erase Sector 7, read base address	Read returns `0xFFFFFFFF`	`0xFFFFFFFF`	Pass
TC3	Flash write/read-back	Write `0xDEADBEEF` to Sector 7 base, read back	Read returns exactly `0xDEADBEEF`	`0xDEADBEEF`	Pass
TC4	Identity provisioning	Run the one-time provisioning routine	Marker + registration + roll + name written; read-back matches what was entered	Matched	Pass
TC5	Identity display at boot	Reset the board after provisioning	Registration number, roll number, and name are displayed first, before anything else	As expected	Pass
TC6	Identity survives testing	Run the Milestone D test suite several times, then reset	Identity block is unchanged - only Sector 7 was touched	As expected	Pass
TC7	Blank results fallback	Erase Sector 7 only, reboot without running a test	System displays “no previous test data” and does not crash	As expected	Pass
TC8	Per-resolution averaging	Trigger a test run; log the raw samples at each resolution alongside the stored average	Stored average voltage matches an independently-computed average of the logged samples for every resolution	Matched	Pass
TC9	Result overwrite on re-test	Run the test suite twice via two separate UART triggers, with the POT moved between runs	Second run's four voltages are the ones shown after the next reset - no mixing of old/new data	As expected	Pass
TC10	Resolution boundary sanity	Inspect stored voltages at all four resolutions for the same POT position	All four are reasonably close to each other (within expected quantisation error), scaling with resolution	Values are very close	Pass
TC11	UART auto-trigger reliability	Send a single keypress, then a rapid burst of keypresses, on separate occasions	Both reliably start exactly one full 4-resolution test run - no missed trigger, no duplicate/overlapping runs	As expected	Pass
TC12	Power-cycle persistence	Unplug/replug USB after both provisioning and testing	Identity and last test results both survive and display correctly on the next boot	As expected	Pass

Identity Provisioning Process

Student identity for both group members is packed into a 32-byte, word-aligned `IdentityBlock_t`: a block-level validity marker (`0xB1010001`), followed by two 64-byte `StudentInfo_t` slots — one per group member.

Provisioning itself erases Sector 6 `(Flash_EraseSector(6))` and then writes the entire identity block — outer marker plus both members' data. 33 words total — in a single `Flash_WriteBlock()` call. First assembling each member's slot in RAM via two calls to `Identity_FillSlot()`.

At every subsequent boot, `Identity_Display()` reads only the marker word at the sector base: a match against `0xB1010001` prints the stored details, while an erased value (`0xFFFFFFFF`) prints a "not yet provisioned" message instead of garbage.

Test-Trigger UX (UART Auto-Trigger)

The four-resolution test suite is not started by a specific command character; any byte received on `USART2 (115200 8N1, PA2/PA3)` is treated as "start test now." The main loop blocks on the first byte, then starts a debounce window during which any further incoming bytes reset the window instead of starting a second, overlapping test run.

This satisfies the requirement that a burst of bytes from a single keypress — which some terminals send as multiple bytes — cannot restart the sequence mid-run. Each trigger runs all four resolutions (12/10/8/6-bit) in sequence, averaging 16 samples per resolution before converting to millivolts and re-erasing/rewriting Sector 7 with the new results.

Analysis Questions

Q1. Why does Flash require an erase-before-write cycle, while RAM does not? Relate your answer to what a single Flash bit-cell physically represents.

Here programming a Flash cell injects electrons onto the floating gate, which can only push a bit from its erased state (logic 1) toward 0. The injection mechanism has no reverse direction at the single-bit level.

Returning a bit to 1 requires physically removing that trapped charge by applying a strong erase voltage across an entire block of cells at once (a sector, in this device), not a single bit or byte.

RAM has no such asymmetry: an SRAM cell is a bistable latch that can be driven to either state directly and immediately by the write circuitry. So `Flash_WriteWord()` can only ever clear bits (1→0) and `Flash_EraseSector()` must run first whenever a word needs to go back to all-1s before being reprogrammed with new data.

Q2. Sector 7 has a finite number of erase cycles (≈10,000). If a user re-triggered the test suite every few seconds all day, estimate how long Sector 7 would last before likely failure. What design change would you make to reduce wear without losing the ability to see recent results?

For example, Triggering every 5 seconds gives 12 runs/minute × 60 minutes × 24 hours = 17,280 erase cycles per day. Now for 10,000 erase cycles of endurance, Sector 7 would be expected to approach failure in roughly 10,000 / 17,280 ≈ 0.58 of a day which is under 14 hours of continuous re-triggering at that rate.

To reduce wear without losing the ability to see recent results, we can only erase-and-rewrite Sector 7 when the new averaged voltages differ from the last stored values by more than a small threshold, or buffer several runs in RAM and flush to Flash on a longer timer/idle period rather than immediately after every trigger.

Q3. Why does this lab store identity and test results in two separate sectors instead of one? What specifically would go wrong if both lived in the same sector?

Because test results are rewritten on every run but identity is written exactly once, sharing a sector would mean every test run's erase-before-write would also wipe the identity block sitting in the same sector — there is no way to erase only the results portion while leaving the identity portion intact.

If both blocks lived in one sector, either every test run would have to also rewrite the untouched identity data immediately after each erase or a bug/omission in that rewrite step would silently and permanently lose the student's registration, roll, and name the very first time the test suite ran.

Q4. Compare the four resolutions' stored voltages for the same POT position. Quantify the expected 1-LSB step size at each resolution and relate it to the differences you observed.

With VREF+ = 3.3 V, the 1-LSB voltage step at each resolution is VREF+ / max_code:

12-bit → 3.3/4095 ≈ 0.806 mV, 8-bit → 3.3/255 ≈ 12.94 mV, and
10-bit → 3.3/1023 ≈ 3.226 mV, 6-bit → 3.3/63 ≈ 52.38 mV per step

For a fixed potentiometer position, the four stored voltages should therefore agree with each other only to within the coarsest resolution's step size — the 6-bit reading can differ from the 12-bit reading by up to roughly half its own 52 mV step (quantisation error), while the 12-bit and 10-bit readings should track each other far more tightly, typically within a few millivolts.

Q5. What is the risk of losing power in the middle of a Flash erase or write operation (e.g. during a test run), and how might a more robust design (briefly, conceptually) protect against a corrupted results block?

If power is lost mid-erase, the sector can be left in an indeterminate state — partially erased, with some words already back to `0xFFFFFFFF` and others still holding old data.

The validity marker itself might land in a state that is neither the expected marker value nor the fully-erased `0xFFFFFFFF` pattern. Because `Identity_Display()/Results_Display()` only checks for those two specific values, a corrupted marker would need explicit handling (as already included) rather than being silently misread as valid data.

A more robust design would avoid overwriting the only valid copy of the results in place, such as keep two result slots in the sector and always write the new result into whichever slot is not the current valid one, then flip a single “active slot” word last, after the rest of the new data is confirmed.

If power is lost during the update, the previously valid slot is still intact and readable, and only the in-progress slot is left corrupted — the boot code would still show the last known-good results instead of garbage.

Q6. Compare your register-level and HAL implementations of `ADC_ReadRaw()`: what does HAL do for you that you had to do manually, and is there any timing/performance cost to that convenience?

The register-level ADC_ReadRaw() directly sets `ADC1_CR2.SWSTART` and polls `ADC1_SR.EOC` before reading `ADC1_DR` — every bit written and every flag checked is explicit. The HAL equivalent hides all of that behind `HAL_ADC_Start()/ HAL_ADC_PollForConversion()/ HAL_ADC_GetValue()`, and behind `HAL_ADC_Init()/ HAL_ADC_ConfigChannel()` at setup time, which internally perform clock enabling and sequencer/sampling-time configuration equivalent to the register-level path.

The convenience comes at a small but real cost: HAL's functions add parameter validation, state-machine bookkeeping (the `ADC_HandleTypeDef's` internal state field), and extra function-call layers before the same register writes ultimately happen, so each HAL call has more overhead per conversion than a direct register write.